



PROJECT: DESIGNING A SERVERLESS DATA ANALYTICS SOLUTION ON AWS.

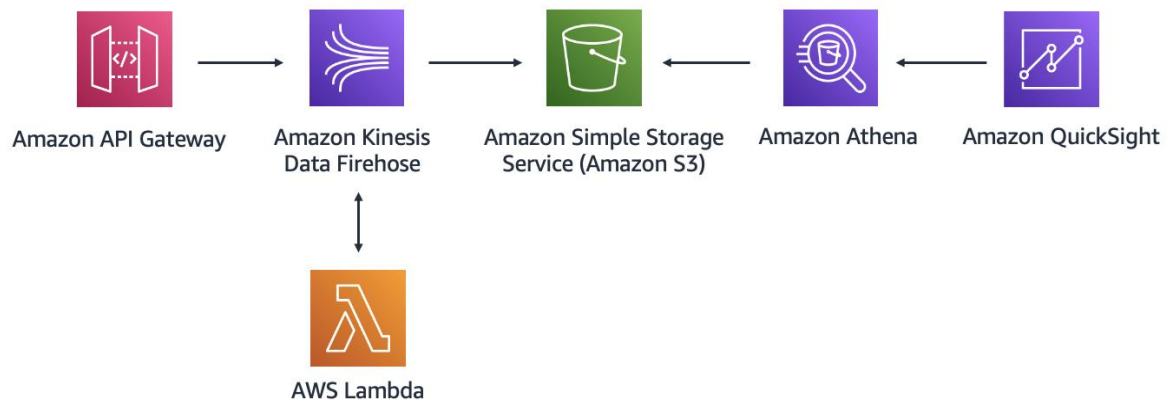
PROJECT AGENDA: In this project, we will design an architecture for a customer who needs an **analytics solution** to **ingest, store, and visualize clickstream data**.

Clickstream data is a **detailed, chronological log** of **every action** a user takes on a **website or app**, acting as a "**digital trail**". It records **pages customers visited, time spent, button clicks, and navigation paths**. **Businesses** use this to **analyze user behavior, improve website design, and personalize experiences**.

Our customer is a fast-food owner who wants to derive insights into all menu items ordered in their **business**.

Our customer is also specific about **data regulatory** concerns and has asked that all data must stay within the **eu-north-1** region.

To achieve this for our customer, we are going to follow the **workflow specified in the AWS architecture diagram stated below**.



1) WHAT IS API GATEWAY AND ITS IMPORTANCE TO OUR PROJECT:

An **API Gateway** is a service gateway that sits between our clients using our website to order food and our backend services, controlling who can access it and where their requests go.

In this **architecture**, we will use an **API Gateway** to ingest customer clickstream data. The API we will create will serve as a **communication gateway** between our application and AWS services.

2) WHAT IS AWS LAMBDA AND ITS IMPORTANCE TO OUR PROJECT:

The **AWS Lambda** is a **function that runs only when triggered**, and you don't manage any **server**; AWS runs it for you. You just write the function code in any programming language you understand, and AWS handles the rest. **You only pay for the seconds it actually runs.**

AWS Lambda is triggered by a **request or event**, which builds a **Lambda function**. The Lambda function runs in a **microVM (micro virtual machine)** and is powered by something called **firecrackers**. This microVM usually shuts down after **15 minutes**.

In this **architecture**, the **Lambda function** will **transform (clean)** the **ingested data** in **Kinesis Data Firehose** or **Amazon Data Firehose** before sending it to our **S3 bucket**.

3) WHAT ARE AWS KINESIS SERVICES AND WHY CHOOSE KINESIS DATA FIREHOSE (AMAZON DATA FIREHOSE):

AWS Kinesis is a platform for **collecting and processing large streams of data in real-time**, and we can think of it as a **conveyor belt** that continuously moves data from **one place** to **another** the moment it arrives.

In this **architecture**, the **Kinesis Firehose delivery stream** will place all the files into an **Amazon S3 bucket**.

THERE ARE FOUR TYPES OF AWS KINESIS SERVICES

TYPE	DEFINITION
Kinesis Data Streams	This type of kinesis collects and stores raw streaming data for custom processing in real time, but coding expertise is needed . If we want to deliver streaming data in real time without any latency , then this type of Kinesis must be used.
Kinesis Data Firehose	This type automatically delivers streaming data to a destination like S3, Redshift , etc and no coding expertise is needed. But a disadvantage is the latency it takes to deliver streaming data to a destination. If latency is not a big cause for concern , this type of Kinesis must be used.
Kinesis Data Analytics	Run SQL queries on streaming data in real-time
Kinesis Video Streams	Stream live video from devices for processing or storage

Our customer has the option of choosing between **Kinesis Data Streams** and **Kinesis Data Firehose**. We chose the latter because our customer does not have a team of **experienced programmers** and does not want the **clickstream data analyzed** in **real time**.

4) WHAT IS THE AMAZON SIMPLE STORAGE SERVICE (S3) AND ITS IMPORTANCE TO OUR PROJECT:

Amazon S3 (Simple Storage Service) is like a giant, secure **online storage locker** that you can access from **anywhere in the world**. One important **characteristic of S3** is its ability to act as a **data lake** and integrate with other services because it is not tied to one.

In this **architecture**, the **S3 bucket** will store **the streaming data** from **Kinesis Data Firehose** in the AWS Cloud.

5) WHAT IS THE AMAZON ATHENA AND ITS IMPORTANCE TO OUR PROJECT:

Amazon Athena is a service that lets you **run SQL queries directly on files sitting in an S3 bucket** without moving the **data anywhere** or **setting up an RDS database**.

Amazon Athena turns your **S3 bucket into a queryable database** without the need for servers, without loading data, and without any database setup. We point it at your S3 bucket files and start writing SQL.

In this **architecture**, **Amazon Athena** will query **data in the S3 bucket for analysis using the Structured Query Language (SQL)**. Remember that this data was ingested into our cloud infrastructure using a REST API from the API gateway into Kinesis Data Firehose, transformed with a lambda function, and then stored in an S3 bucket.

6) WHAT IS AMAZON QUICKSIGHT AND ITS IMPORTANCE TO OUR PROJECT:

Amazon QuickSight is a **cloud-based business intelligence (BI)** service that turns **raw data** into **easy-to-understand, interactive, and visual dashboards**. You can think of it as a tool that **creates smart charts and reports** from your **company's data**, allowing you to "see" **trends, outliers, and insights** from your **company's data**.

In this **architecture**, **Amazon QuickSight** will allow us to get better insights from our **streaming data** from Data FireHose, published in S3, and queried in Amazon Athena.

Amazon QuickSight is a **subscription service** and can be used for free for the first **30 days**. You need to delete your **QuickSight account** after you complete this exercise.

STEP 1: CREATE IAM POLICY THAT WILL GRANT PERMISSIONS TO WRITE RECORDS TO AMAZON KINESIS DATA FIREHOSE DELIVERY STREAM.

I) NAVIGATE TO **IAM** AND CLICK ON **POLICY**, AND CLICK ON **CREATE A NEW POLICY**.

II) IN THE **SPECIFY PERMISSIONS PAGE**, SWITCH TO THE **JSON TAB** AND **REPLACE THE DEFAULT PLACEHOLDER CODE** BY PASTING **THE POLICY** BELOW AND CLICK **NEXT**.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "VisualEditor0",
      "Effect": "Allow",
      "Action": "firehose:PutRecord",
      "Resource": "*"
    }
  ]
}
```

III) GIVE YOUR **POLICY** THE NAME **“API-Firehose”**, ADD A **DESCRIPTION** **“This policy allows Kinesis data firehose to receive data”**, AND CLICK ON **CREATE POLICY**.

IV) THE **API-Firehose POLICY** WILL **GRANT PERMISSIONS FOR WRITE RECORDS TO AMAZON KINESIS DATA FIREHOSE DELIVERY STREAM**.

STEP 2: CREATE AN IAM ROLE THAT ENABLES API GATEWAY TO SEND STREAMING DATA FROM THE CLICKSTREAM DATA TO AMAZON KINESIS DATA FIREHOSE. YOU WILL THEN ATTACH THE API-Firehose POLICY THAT YOU CREATED TO THE ROLE.

I) NAVIGATE TO **IAM** AND CLICK ON **ROLE**, AND CLICK ON **CREATE A NEW ROLE**.

II) UNDER **TRUSTED ENTITY TYPE**, SELECT **AWS SERVICE**, AND UNDER **USE CASE** SELECT **API GATEWAY** AND CLICK ON **NEXT** AND CLICK ON **NEXT AGAIN**.

III) GIVE YOUR **ROLE** THE NAME **“APIGateway-Firehose.”** REVIEW YOUR CONFIGURATIONS AND CLICK ON **CREATE ROLE**.

IV) SEARCH FOR **APIGateway-Firehose**, CLICK ON IT, CLICK ON **ADD PERMISSIONS**, AND CLICK ON **ATTACH POLICIES**.

V) SEARCH FOR **API-Firehose POLICY** AND **CLICK ON THE CHECK BOX**, AND CLICK ON **ADD PERMISSIONS**. OBSERVE THAT WE NOW HAVE **AN AWS MANAGED POLICY** AND **A CUSTOMER MANAGED POLICY** ATTACHED TO THE **APIGateway-Firehose ROLE**.

STEP 3: CREATE AN S3 BUCKET TO STORE DATA.

I) SELECT THE **eu-north-1** REGION.

II) SEARCH FOR **S3 BUCKET**. NOTE THAT YOUR **S3 BUCKET NAME** MUST BE **GLOBALLY UNIQUE**. FOR ME, I WILL USE THE BUCKET NAME **data-lake-1400-bucket**.

III) LEAVE ALL THE **DEFAULT CONFIGURATIONS** AND CLICK ON **CREATE A BUCKET**.

IV) AFTER CREATING YOUR BUCKET, CLICK ON YOUR **BUCKET NAME URL**, AND CLICK ON THE **PROPERTIES TAB**. LOCATE THE **BUCKET ARN**, **COPY IT**, AND **PASTE IT** IN A SEPARATE SHEET. WE ARE GOING TO USE IT IN THE FUTURE.

YOUR ARN MIGHT LOOK LIKE `arn:aws:s3:::data-lake-1400-bucket`

STEP 4: CREATE THE LAMBDA FUNCTION TO TRANSFORM DATA.

I) MAKE SURE YOU ARE STILL IN THE **eu-north-1** REGION.

II) SEARCH FOR **LAMBDA**, AND IN THE **LAMBDA DASHBOARD**, CLICK ON **CREATE A FUNCTION**.

III) IN THE **CREATE FUNCTION PAGE**, SELECT USE A **BLUEPRINT**.

IV) UNDER **BASIC INFORMATION**, LOCATE **BLUEPRINT NAME**, CLICK ON THE DROP-DOWN, AND SEARCH FOR **PROCESS RECORDS SENT TO AN AMAZON DATA FIREHOSE STREAM PYTHON 3.12**. NOTE THAT THE **PYTHON VERSION** USED IS THE **LATEST PYTHON VERSION RELEASE** AND MIGHT CHANGE TO THE **LATEST** WHEN YOU ARE **ATTEMPTING THIS PROJECT**.

V) UNDER **FUNCTION NAME**, GIVE YOUR FUNCTION ANY NAME. IN MY CASE, I WILL GIVE MY FUNCTION THE NAME **transform-data-firehose-data**

VI) LEAVE ALL THE OTHER DEFAULTS AND CLICK ON **CREATE FUNCTION**.

VII) YOU WILL BE **AUTOMATICALLY TAKEN** TO YOUR **FUNCTION PAGE**, WHICH IS THE NAME YOU GAVE YOUR FUNCTION, FOR EXAMPLE, **transform-data-firehose-data**.

IX) LOCATE THE **CODE TAB**, AND UNDER **EXPLORER**, IN YOUR **lambda_function.py** FILE, REPLACE THE DEFAULT PYTHON SCRIPT WITH THE ONE BELOW, OR USE THE **transform-data-firehose-data.py** PYTHON FILE.

```
import base64

def lambda_handler(event, context):
    output = []

    for record in event['records']:
        payload = base64.b64decode(record['data']).decode('utf-8')
        row_w_newline = payload + "\n"
        row_w_newline = base64.b64encode(row_w_newline.encode('utf-8')).decode('utf-8')
        output_record = {
            'recordId': record['recordId'],
            'result': 'Ok',
            'data': row_w_newline
        }
        output.append(output_record)

    return {'records': output}
```

The sole job of the Python script above is to ensure each record in the delivery stream ends **with a newline** so that when the **data lands in S3**, each record sits on its **own line** and is **readable**. Without this, our data will be **complex to read**, and **Lambda** is the perfect service for that since our customers are inputting their data at **different intervals** through our **clickstream** data service.

X) LOCATE **DEPLOY** UNDER **EXPLORER** AND CLICK ON **DEPLOY**, AND YOU SHOULD SEE THE MESSAGE **SUCCESSFULLY UPDATED THE FUNCTION transform-data-firehose-data**.

XI) LOCATE THE **CONFIGURATION TAB** AND IN THE **GENERAL CONFIGURATION PAGE**, CLICK ON **EDIT**.

XII) LOCATE THE **TIMEOUT** AND INCREASE THE **TIMEOUT** TO **10 SECONDS** TO PREVENT OUR FUNCTION FROM **TIMING OUT** IF WE HAVE A HIGH TURN OF EVENTS WE ARE TRYING TO PROCESS. CLICK ON **SAVE**.

XIV) SCROLL UP, LOCATE YOUR **FUNCTION'S ARN**, AND COPY THE **FUNCTION ARN**. PASTE IT IN A **SEPARATE SHEET**. WE ARE GOING TO USE IT IN THE FUTURE.

YOUR ARN MIGHT LOOK LIKE `arn:aws:lambda:eu-north-1:140023390772:function:transform-data-firehose-data`

STEP 5: CREATE A KINESIS DATA FIREHOSE TO INGEST DATA.

I) MAKE SURE YOU ARE STILL IN THE **eu-north-1** REGION.

II) SEARCH FOR **KINESIS**, IN THE **KINESIS DASHBOARD**, UNDER **GET STARTED**, CLICK ON **AMAZON DATA FIREHOSE – NEW**, AND CLICK ON **CREATE FIREHOSE STREAM**.

III) IN THE **CREATE FIREHOSE STREAM** PAGE, UNDER **CHOOSE SOURCE AND DESTINATION**, UNDER **SOURCE**, SELECT **DIRECT PUT** BECAUSE WE WANT TO PUT RECORDS DIRECTLY FROM

API GATEWAY INTO **KINESIS DATA FIREHOSE**. AND UNDER DESTINATION SELECT **AMAZON S3** BECAUSE THAT IS WHERE WE WILL STORE OUR DATA.

IV) UNDER **FIREHOSE STREAM NAME**, YOU CAN RENAME YOUR STREAM OR LEAVE IT AT THE DEFAULT. FOR ME, I WILL RENAME MY STREAM TO **click-stream-api-gateway-data-ingestion**

V) LOCATE, **TRANSFORM, AND CONVERT RECORDS – OPTIONAL**, AND UNDER **TRANSFORM SOURCE RECORDS WITH AWS LAMBDA**, CLICK ON THE **TURN ON DATA TRANSFORMATION CHECK BOX**. UNDER **AWS LAMBDA FUNCTION**, COPY AND PASTE THE **ARN** OF THE **transform-data-firehose-data** LAMBDA FUNCTION WE CREATED EARLIER, AND UNDER **VERSION OR ALIAS**, SELECT **LATEST**.

VI) LOCATE **DESTINATION SETTINGS** AND UNDER **S3 BUCKET**, CLICK ON **BROWSE** AND SELECT THE BUCKET YOU CREATED FOR THIS PROJECT. FOR ME, I WILL USE THE **data-lake-1400-bucket** I CREATED FOR THIS PROJECT AND CLICK ON **CHOOSE**.

VII) LEAVE THE REMAINING DEFAULTS AND CLICK ON **CREATE FIREHOSE STREAM**.

VIII) IN YOUR KINESIS DATA FIREHOSE STREAM **click-stream-api-gateway-data-ingestion** PAGE, COPY THE **ARN** OF YOUR **KINESIS DATA FIREHOSE STREAM** AND PASTE IN A SEPARATE SHEET. WE WILL USE IT IN THE FUTURE.

ALSO, YOU CAN WAIT FOR THE STATUS OF YOUR **KINESIS DATA STREAM** TO CHANGE FROM **CREATING** TO **ACTIVE**, WHICH MIGHT TAKE SEVERAL MINUTES.

IX) AFTER THE STATUS OF YOUR **KINESIS DATA STREAM** HAS CHANGED TO **ACTIVE**, SCROLL DOWN AND CLICK ON THE **CONFIGURATION TAB**. IN THE CONFIGURATION TAB, SCROLL DOWN AND LOCATE **SERVICE ACCESS**, AND OPEN THE **IAM ROLE** IN ANOTHER TAB. COPY THE **ARN** OF THAT ROLE IN A SEPARATE SHEET.

WE WILL USE THIS ARN TO MODIFY THE **BUCKET POLICY** OF OUR S3 BUCKET.

X) NAVIGATE BACK TO S3 BY **SEARCHING FOR S3** IN THE **SEARCH BAR**, AND SELECT THE BUCKET YOU CREATED.

XI) LOCATE THE **PERMISSIONS TAB**, SELECT **BUCKET POLICY**, AND CLICK ON **EDIT**.

XII) PASTE THE **BUCKET POLICY** BELOW IN THE **BUCKET POLICY EDITOR** AFTER MAKING THE NECESSARY CHANGES BY INPUTTING THE **ARN** OF YOUR **KINESIS DATA FIREHOSE ROLE** AND THE **ARN** OF YOUR **S3 BUCKET**. ALTERNATIVELY YOU CAN CHECK THE **s3_bucket_policy.json** FILE. CLICK ON **SAVE CHANGES**.

```
{
  "Version": "2012-10-17",
  "Id": "PolicyID",
  "Statement": [
    {
      "Sid": "StmntID",
      "Effect": "Allow",
      "Principal": {
        "AWS": "<Enter the ARN for the Kinesis Firehose IAM role>"
      },
      "Action": [
        "s3:AbortMultipartUpload",
        "s3:GetBucketLocation",
        "s3:GetObject",
        "s3:ListBucket",
        "s3:ListBucketMultipartUploads",
        "s3:PutObject"
      ],
      "Resource": [
        "<Enter the ARN of the S3 bucket>",
        "<Enter the ARN of the S3 bucket>/*"
      ]
    }
  ]
}
```

STEP 6: CREATE API GATEWAY TO INGEST CUSTOMER CLICKSTREAM DATA FROM OUR APPLICATION.

I) MAKE SURE YOU ARE STILL IN THE **eu-north-1** REGION.

II) SEARCH FOR **API GATEWAY**, AND IN THE **API DASHBOARD**, CLICK ON **CREATE AN API**.

III) IN THE **CHOOSE AN API TYPE PAGE**, SELECT **REST API** BY CLICKING **BUILD**.

THE **REST API** SUPPORTS **DATA TRANSFORMATION** USING A **VELOCITY TEMPLATE LANGUAGE (VTL)**, WHICH ENABLES THE **CLICKSTREAM DATA STRUCTURE** TO FIT OUR **KINESIS DATA FIREHOSE SERVICE DATA STRUCTURE**.

IV) IN THE **CREATE REST API PAGE**, UNDER API DETAILS, SELECT **NEW API**.

V) UNDER **API NAME**, GIVE YOUR API ANY NAME. FOR ME, I WILL GIVE MY API THE NAME **ingest-data-from-clickstream-into-kinesis-data-firehose**. LEAVE THE DEFAULT CONFIGURATIONS AND CLICK ON **CREATE API**.

VI) IN THE **RESOURCE PAGE**, CLICK ON **CREATE RESOURCE** AND UNDER **RESOURCE NAME**, GIVE YOUR RESOURCE THE NAME **url-link**, AND CLICK ON **CREATE RESOURCE**.

THE **RESOURCE PAGE** IS WHAT GENERATES A **URL LINK** FOR THE **API GATEWAY** THAT THE **CLICKSTREAM DATA** CAN USE TO INPUT DATA INTO **KINESIS DATA FIREHOSE**.

VII) IN THE **RESOURCES PAGE**, UNDER **METHODS**, CLICK ON **CREATE METHOD**.

VIII) IN THE **CREATE METHOD PAGE**, UNDER **METHOD TYPE**, SELECT **POST**.

UNDER **INTEGRATION TYPE**, SELECT **AWS SERVICE**.

UNDER **AWS REGION**, SELECT **eu-north-1**.

UNDER **AWS SERVICE**, SELECT **firehose**.

UNDER **AWS SUBDOMAIN**, LEAVE EMPTY.

UNDER **HTTP METHOD**, SELECT **POST** BECAUSE WE WANT THIS API TO POST THE **CLICKSTREAM DATA** INTO **KINESIS DATA FIREHOSE**.

UNDER **ACTION TYPE**, CHOOSE **USE ACTION NAME**.

UNDER **ACTION NAME**, USE THE NAME **PutRecord**.

UNDER **EXECUTION ROLE**, **PASTE** THE **ARN** OF THE **APIGateway-Firehose ROLE** WE CREATED INITIALLY. IT SHOULD LOOK LIKE `arn:aws:iam::140023390772:role/APIGateway-Firehose`

LEAVE THE REMAINING DEFAULT CONFIGURATIONS AND CLICK ON **CREATE METHOD**.

IX) WE WANT TO MATCH THE **INCOMING CLICKSTREAM DATA FORMAT** WITH THE **DATA FORMAT** WE HAVE IN **KINESIS DATA FIREHOSE**. WITHOUT THIS, **KINESIS DATA FIREHOSE** WILL REJECT THE INCOMING REQUEST.

IN THE **RESOURCES DASHBOARD**, LOCATE THE **INTEGRATION REQUEST TAB** AND CLICK ON **EDIT**. LOCATE **MAPPING TEMPLATES**, CLICK ON THE DROP-DOWN, AND CLICK ON **ADD MAPPING TEMPLATE**.

UNDER **CONTENT TYPE**, TYPE IN **application/json**, UNDER **GENERATE TEMPLATE TAB**, LEAVE IT EMPTY, AND IN THE **TEMPLATE BODY**, COPY AND PASTE THE **VELOCITY TEMPLATE LANGUAGE** FILE BELOW. MAKE SURE YOU INPUT YOUR **KINESIS DATA FIREHOSE NAME**, FOR EXAMPLE, **click-stream-api-gateway-data-ingestion**, WHICH IS THE NAME OF MY DELIVERY STREAM. CLICK **SAVE**.

LOCATE **REQUEST BODY PASSTHROUGH**, SELECT **When there are no templates defined (recommended)**

```
{
  "DeliveryStreamName": "<Enter the name of your delivery stream>",
  "Record": {
    "Data":
"$util.base64Encode($util.escapeJavaScript($input.json('$')).replace('\n', ''))"
  }
}
```

X) WE WANT TO **SIMULATE THE EVENTS** AS IF THEY WERE COMING FROM **OUR CLICKSTREAM APPLICATION**.

IN THE **RESOURCE PAGE**, **SCROLL DOWN** AND BY YOUR **FAR RIGHT LOCATE THE TEST TAB** AND CLICK ON IT. IN THE REQUEST BODY, COPY AND PASTE THE JSON SCRIPT BELOW AND CLICK ON **TEST**.

```
{  
  "element_clicked": "entree_1",  
  "time_spent": 12,  
  "source_menu": "restaurant_name",  
  "created_at": "2025-09-11 23:00:00"  
}
```

REVIEW THE BODY OF THE **REQUEST LOGS**, AND WE SHOULD SEE THE MESSAGE: **status 200** AT THE TOP, WHICH MEANS **API GATEWAY PROCESSED OUR REQUEST SUCCESSFULLY**.

ALSO IN THE MIDDLE OF THE **REQUEST LOGS**, YOU SHOULD SEE THE BELOW:

Sun Mar 15 10:19:41 UTC 2026 : Sending request to https://firehose.eu-north-1.amazonaws.com/?Action=PutRecord

Sun Mar 15 10:19:41 UTC 2026 : Received response. Status: 200, Integration latency: 37 ms

STATUS 200 MEANS THAT THE **TEST DATA** WAS SUCCESSFULLY **TRANSFORMED INTO KINESIS DATA FIREHOSE**. ANY OTHER **STATUS CODE** MEANS THAT THE **TRANSFORMATION INTO KINESIS DATA FIREHOSE** FROM WAS **UNSUCCESSFUL**.

IX) REMOVE THE **JSON SCRIPT ABOVE** AND PASTE THE **JSON SCRIPT BELOW** TO TEST FOR A NEW ENTRY. CLICK ON **TEST**.

```
{  
  "element_clicked": "entree_4",  
  "time_spent": 32,  
  "source_menu": "restaurant_name",  
  "createdAt": "2025-09-11 23:00:00"  
}
```

REVIEW THE **REQUEST LOGS**, AND WE SHOULD ALSO SEE THE MESSAGE: **status 200** AT THE TOP, WHICH MEANS **API GATEWAY** PROCESSED OUR **REQUEST SUCCESSFULLY**.

X) REMOVE THE **JSON SCRIPT ABOVE** AND PASTE THE **JSON SCRIPT BELOW** TO TEST FOR A NEW ENTRY. CLICK ON **TEST**

```
{
  "element_clicked":"drink_1",
  "time_spent":15,
  "source_menu":"restaurant_name",
  "created_at":"2022-09-11 23:00:00"
}
```

REVIEW THE **REQUEST LOGS** AGAIN, AND WE SHOULD ALSO SEE THE MESSAGE: **status 200** AT THE TOP, WHICH MEANS **API GATEWAY** PROCESSED OUR **REQUEST SUCCESSFULLY**.

XI) REMOVE THE **JSON SCRIPT ABOVE** AND PASTE THE **JSON SCRIPT BELOW** TO TEST FOR A NEW ENTRY. CLICK ON **TEST**

```
{
  "element_clicked":"drink_3",
  "time_spent":14,
  "source_menu":"restaurant_name",
  "created_at":"2022-09-11 23:00:00"
}
```

REVIEW THE **REQUEST LOGS** AGAIN, AND WE SHOULD ALSO SEE THE MESSAGE: **status 200** AT THE TOP, WHICH MEANS **API GATEWAY** PROCESSED OUR **REQUEST SUCCESSFULLY**.

XII) TO CONFIRM THAT OUR DATA WAS **SUCCESSFULLY TRANSFORMED** AND PUSHED INTO **API GATEWAY**, LOCATE YOUR CREATED **S3 BUCKET** USED FOR THIS PROJECT, CLICK ON **OBJECTS**, AND YOU SHOULD SEE A FOLDER CALLED **2026/**. THE NAME OF THIS FOLDER WILL BE DETERMINED BY THE **YEAR YOU CREATED THIS FOLDER**.

NOTE THAT THERE WILL BE **LATENCY** FROM WHEN THE **DATA IS PUSHED** FROM THE **API GATEWAY** TO **KINESIS DATA FIREHOSE** AND TO YOUR **S3 BUCKET**, SO YOU NEED TO KEEP **REFRESHING** YOUR **OBJECTS** IN YOUR **S3 BUCKET** TO SEE THE **TRANSFORMED DATA** FROM **KINESIS DATA FIREHOSE**.

IMPORTANT NOTE: IF YOU ENCOUNTER AN ERROR FROM THE TEMPLATE MAPPING, DO NOT EDIT THE TEMPLATE MAPPING. REMOVE THE OLD TEMPLATE MAPPING AND CREATE A NEW TEMPLATE MAPPING.

STEP 7: CREATE AN ATHENA TABLE TO QUERY S3 DATA WITH SQL

I) MAKE SURE YOU ARE STILL IN THE **eu-north-1** REGION.

II) SEARCH FOR **ATHENA** IN THE SEARCH BAR AND NAVIGATE TO THE **ATHENA DASHBOARD**.

III) IN THE ATHENA DASHBOARD, SELECT **QUERY YOUR DATA WITH ATHENA CONSOLE** AND CLICK ON **LAUNCH QUERY EDITOR**.

IV) WE WANT TO CREATE A **PREFIX (A SUB-FOLDER INSIDE A BUCKET)** TO STORE THE **QUERY RESULTS FROM ATHENA AS AN OBJECT INSIDE THIS PREFIX (SUB-FOLDER)** TO ORGANIZE THE OBJECTS IN OUR S3 BUCKET. WE WILL CREATE A PREFIX FROM THE **S3 BUCKET WE CREATED FOR THIS PROJECT**.

IN THE **QUERY EDITOR**, CLICK ON THE **QUERY SETTINGS TAB** AND SELECT **MANAGE**.

V) IN THE **MANAGE QUERY RESULT LOCATION AND ENCRYPTION** POP-UP PAGE, LOCATE **BROWSE S3**, CLICK ON IT, AND SELECT THE **S3 BUCKET YOU CREATED FOR THIS PROJECT**. THE NAME OF MY OWN BUCKET IS **data-lake-1400-bucket**.

VI) AFTER SELECTING YOUR BUCKET, APPEND **/results/** AT THE END OF YOUR **BUCKET NAME** TO CREATE A PREFIX (SUB-FOLDER) TO STORE THE QUERY RESULTS FROM ATHENA. YOUR S3 BUCKET NAME SHOULD NOW LOOK LIKE **s3://data-lake-1400-bucket/results/**

VII) CLICK **SAVE** AND CLICK ON THE **EDITOR TAB**.

VIII) IN THE **EDITOR TAB**, SCROLL DOWN AND LOCATE **TABLE AND VIEWS**, CLICK ON THE **CREATE DROP-DOWN** AND SELECT **CREATE TABLE AS SELECT**.

IX) IN THE **QUERY EDITOR**, REMOVE THE **DEFAULT CODE**, **COPY**, AND **PASTE** IN THE **SQL SCRIPT** BELOW.

```
CREATE EXTERNAL TABLE my_ingested_clickstream_data (  
    element_clicked STRING,  
    time_spent INT,  
    source_menu STRING,  
    created_at STRING  
)  
PARTITIONED BY (  
    datehour STRING  
)  
ROW FORMAT SERDE 'org.openx.data.jsonserde.JsonSerDe'  
WITH SERDEPROPERTIES (  
    'paths' = 'element_clicked, time_spent, source_menu, created_at'  
)  
LOCATION 's3://<Enter your Amazon S3 bucket name>/'  
TBLPROPERTIES (  
    'projection.enabled' = 'true',  
    'projection.datehour.type' = 'date',  
    'projection.datehour.format' = 'yyyy/MM/dd/HH',  
    'projection.datehour.range' = '2021/01/01/00,NOW',  
    'projection.datehour.interval' = '1',  
    'projection.datehour.interval.unit' = 'HOURS',  
    'storage.location.template' = 's3://<Enter your Amazon S3 bucket  
name>/${datehour}/'  
)
```


X) IN **LINE 14**, LOCATE **LOCATION 's3://<Enter your Amazon S3 bucket name>/'** AND REPLACE **<Enter your Amazon S3 bucket name>** WITH THE **S3 BUCKET NAME** YOU CREATED FOR THIS PROJECT. IT SHOULD LOOK LIKE **'s3://data-lake-1400-bucket/'**

XI) IN **LINE 22**, LOCATE **'storage.location.template' = 's3://<Enter your Amazon S3 bucket name>/\${datehour}/'** AND REPLACE **<Enter your Amazon S3 bucket name>** WITH THE **S3 BUCKET NAME** YOU CREATED FOR THIS PROJECT. IT SHOULD LOOK LIKE **'s3://data-lake-1400-bucket/'**

X) CLICK ON **RUN**, AND YOU SHOULD HAVE A COMPLETED MESSAGE WITH A **QUERY SUCCESSFUL MESSAGE** IF WE SUCCESSFULLY CREATED THE **my_ingested_clickstream_data** TABLE IN ATHENA.

LOCATE **TABLES** IN THE **QUERY EDITOR**, AND YOU SHOULD SEE THE **my_ingested_clickstream_data** TABLE WE JUST CREATED. CLICK ON THE **+ ICON**, AND WE SHOULD SEE THE **COLUMNS CREATED INSIDE THIS TABLE**.

XI) OPEN A **NEW QUERY TAB** BY CLICKING ON THE **+ ICON** AT THE **FAR RIGHT** OF YOUR CURRENT QUERY TAB. TYPE THE **SQL QUERY BELOW** AND CLICK ON THE **RUN BUTTON**. THE RESULTS SHOULD SHOW OUR **INPUTS** FROM THE **AMAZON API GATEWAY** IN THE **my_ingested_clickstream_data** TABLE WE JUST CREATED.

```
SELECT * FROM my_ingested_clickstream_data;
```

STEP 8: CREATE A QUICKSIGHT DASHBOARD TO VISUALIZE DATA WITH CHARTS

I) MAKE SURE YOU ARE STILL IN THE **eu-north-1** REGION.

II) SEARCH FOR **QUICKSIGHT**.

III) FOR **NEW USERS**, CLICK ON **SIGN UP FOR QUICKSIGHT**.

IV) IN THE **SIGN-UP FOR AMAZON QUICK** PAGE, LOCATE **ACCOUNT INFORMATION**, UNDER **ACCOUNT NAME**, INPUT THE NAME OF YOUR ACCOUNT, FOR EXAMPLE, **Chinedu**, AND UNDER **EMAIL FOR ACCOUNT NOTIFICATIONS**, INPUT THE EMAIL OF YOUR ACCOUNT, FOR EXAMPLE, onyemachinedu45@gmail.com

V) LOCATE **DEFAULT REGION** AND UNDER **REGION**, SELECT THE REGION YOU WANT YOUR **QUICKSIGHT DATA** TO BE STORED. FOR ME, I WILL SELECT **Europe(Stockholm)**.

VI) UNDER **AUTHENTICATION METHOD AND ENCRYPTION**, LEAVE AT THE DEFAULTS AND CLICK ON **CREATE ACCOUNT**. IF SUCCESSFUL, YOU SHOULD SEE AN **ACCOUNT CREATED SUCCESSFUL** MESSAGE AND THE **USER NAME** (Chinedu) OF WHERE THAT ACCOUNT WAS CREATED.

CLICK ON **MANAGE QUICK**.

VII) IN THE **MANAGE ACCOUNT DASHBOARD**, UNDER **SHORTCUTS**, SELECT **AWS RESOURCES (Allow connections to first-party services)**.

VIII) IN THE **QUICK ACCESS TO AWS SERVICES PAGE**, UNDER **IAM ROLE**, LEAVE AT THE DEFAULT AND UNDER **ALLOW ACCESS AND AUTODISCOVERY FOR THESE RESOURCE** LOCATE **AMAZON S3**, CLICK ON IT, AND IN THE **SELECT AMAZON S3 BUCKETS** POP-UP PAGE, UNDER THE **S3 BUCKETS LINKED TO QUICK ACCOUNT**, SELECT THE **S3 BUCKET YOU CREATED FOR THIS PROJECT**, MINE IS CALLED **data-lake-1400-bucket**, AND CLICK ON **FINISH**.

BACK IN THE **QUICK ACCESS TO AWS SERVICES PAGE**, **SCROLL DOWN** AND CLICK ON **SAVE**.

IX) BACK IN THE **MANAGE ACCOUNT DASHBOARD**, LOCATE THE **EXPLORE HAMBURGER ICON** AT THE TOP, FAR LEFT, AND CLICK ON IT.

X) UNDER THE **QUICKSIGHT DROPDOWN**, CLICK ON THE **ANALYSES TAB**, CLICK ON **CREATE ANALYSIS**, AND IN THE **CREATE ANALYSIS DASHBOARD**, CLICK ON **CREATE DATASET**, IN THE **CREATE DATASET DASHBOARD**, CLICK ON **CREATE DATA SOURCE**.

XI) IN THE **CREATE DATA SOURCE DASHBOARD TAB**, SELECT **AMAZON ATHENA**, CLICK ON **NEXT**, IN THE **NEW AMAZON ATHENA DATA SOURCE**, UNDER **DATA SOURCE NAME**, GIVE IT ANY NAME, FOR ME I WILL NAME MINE **clickstream_data**, AND CLICK ON **CREATE DATA SOURCE**.

XII) IN THE **CHOOSE YOUR TABLE DASHBOARD TAB**, SELECT THE **my_ingested_clickstream_data** SQL TABLE WE CREATED IN ATHENA AND CLICK ON **SELECT**.

XIV) IN **FINISH DATASET CREATION DASHBOARD TAB**, LEAVE THE DEFAULT AND CLICK ON **VISUALIZE**, AND YOU WILL BE DIRECTED TO THE **my_ingested_clickstream_data** ANALYSIS PAGE WHERE YOU CAN START ANALYZING YOUR DATA.

YOU CAN SEE THE **COLUMNS** IN THE **my_ingested_clickstream_data** TABLE AND THE **DIFFERENT CHARTS** WE CAN CHOOSE FROM TO **VISUALIZE OUR DATA** UNDER **VISUALS**.

YOU CAN JUST PLAY AROUND WITH IT. IF YOU WANT TO LEARN MORE ABOUT HOW TO USE AMAZON QUICKSIGHT TO CREATE INTERACTIVE DASHBOARDS, VISIT THE LINK BELOW.

[CREATE VISUALS WITH AMAZON QUICKSIGHT](#)

STEP 9: DELETE ALL THE AWS SERVICES USED TO CREATE YOUR INFRASTRUCTURE.

I) IN **AMAZON QUICKSIGHT**, CLICK ON THE **EXPLORE ICON**, LOCATE **QUICK SIGHT**, AND SELECT **DATASET**. CLICK ON THE **ACTION** AND SELECT **DELETE**.

SELECT THE **DATA SOURCES** TAB, CLICK ON **ACTIONS**, AND CLICK ON **DELETE**.

IF YOU WON'T BE USING QUICKSIGHT IN THE FUTURE, YOU CAN DELETE YOUR ACCOUNT.

A) AT THE FAR RIGHT OF THE **WELCOME TO QUICK** DASHBOARD, LOCATE THE USER ICON, CLICK ON IT, CLICK ON **MANAGE QUICK**, IN THE MANAGE ACCOUNT DASHBOARD, LOCATE ACCOUNT BY THE FAR LEFT, AND CLICK ON **ACCOUNT SETTINGS**, AND UNDER **ACCOUNT TERMINATION**, CLICK ON **MANAGE**.

B) IN THE **ACCOUNT TERMINATION** PAGE, **TURN OFF ACCOUNT TERMINATION PROTECTION**, CONFIRM, AND CLICK ON **DELETE ACCOUNT**.

II) DELETE YOUR S3 BUCKET.

III) DELETE AMAZON ATHENA TABLE

IV) DELETE API GATEWAY

V) DELETE AMAZON DATA FIREHOSE

VI) DELETE AWS LAMBDA FUNCTION

TECH DEALER

DELIVERING TOMORROW'S TECH TODAY